FULL-STACK WEB DEVELOPMENT GUIDE

Building a Full-Stack Mini App

Learn how React talks to Flask one step at a time through a Study Tip Generator

1. What Are We Building & Architecture Overview

We are constructing a client-server mini application where a student enters their name on a React frontend interface, sends an asynchronous HTTP POST request, and receives a personalized study tip generated by a Flask backend.

1. React UI
Student types name in input field

→

2. Fetch API
POST JSON payload to Flask

→

3. Flask API
Processes request & constructs tip

→

4. React Display
Renders tip string dynamically

2. Project Structure & Prerequisites

Ensure you have **Python 3**, **Node.js**, and a code editor (e.g., VS Code) installed. Organize your directory into clear frontend and backend folders:

```
my-app/
├── backend/
│   ├── app.py
│   └── requirements.txt
└── frontend/
    ├── package.json
    └── src/
        └── App.js
```

```
# Environment Setup Commands
$ cd backend
$ pip install flask flask-cors

$ cd ../frontend
$ npx create-react-app .
$ npm start
```

3. Step 1: Building the Flask Backend

Flask is a lightweight Python web framework used here to expose an API endpoint (`/api/tip`).

backend/app.py

```python
from flask import Flask, request, jsonify
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # Enables Cross-Origin Resource Sharing
```

```python
@app.route('/api/tip', methods=['POST'])
def get_tip():
    data = request.get_json()
    name = data.get('name', 'Student')
    tip = f"Hey {name}, try the Pomodoro method today!"
    return jsonify({'tip': tip})

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

- **Route & Method:** `@app.route('/api/tip', methods=['POST'])` defines the HTTP endpoint path and restricts it to POST actions.
- **JSON Processing:** `request.get_json()` parses payload data; `jsonify()` serializes Python dictionaries into standard JSON API responses.
- **CORS Support:** `flask-cors` permits secure browser requests from React (port 3000) to Flask (port 5000).

4. Testing API & Step 2: Building React Frontend

Test endpoint using cURL before frontend integration:

```
$ curl -X POST http://127.0.0.1:5000/api/tip -H "Content-Type: application/json" -d '{"name": "Asha"}'
{"tip": "Hey Asha, try the Pomodoro method today!"}
```

frontend/src/App.js

```javascript
import React, { useState } from 'react';

function App() {
  const [name, setName] = useState('');
  const [tip, setTip] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    const response = await fetch('http://127.0.0.1:5000/api/tip', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name })
    });
    const data = await response.json();
    setTip(data.tip);
  };

  return (
    <div style={{ padding: '20px' }}>
      <h2>Study Tip Generator</h2>
      <form onSubmit={handleSubmit}>
        <input type="text" value={name} onChange={(e) => setName(e.target.value)}
placeholder="Enter your name..." />
        <button type="submit">Get My Tip</button>
      </form>
      {tip && <p style={{ marginTop: '15px' }}>{tip}</p>}
    </div>
```

```
  );
}
```

5. Troubleshooting & Debugging Checklist

- **Flask Server Not Running:** Ensure `python app.py` is actively listening in terminal before submitting from React.
- **CORS Blocked Error:** Verify `CORS(app)` is initialized in Flask to permit requests from origin `http://localhost:3000`.
- **Network 404 / Port Mismatch:** Confirm standard fetch URL specifies port `5000` and route path `/api/tip`.

STUDENT PRACTICE WORKSHEET

Student Name: _____ **Date:** _____

Topic: Full-Stack React & Flask Integration (4th Grade Level Practice)

Question 1: Frontend vs. Backend (Easy Matching & Drawing)

The **Frontend** is what you see on the screen (like buttons and text boxes). The **Backend** is like the brain working behind the scenes.

Task: Draw a line to match each part to where it belongs:

- A blue "Get My Tip" Button ——— **[Backend Brain]**
- Thinking of a helpful study tip ——— **[Frontend Screen]**

Question 2: Sending a Message (Fill in the Blank)

When you type your name on the computer and press the button, your browser sends a message to the server.

Fill in the missing word using these options: **(Name / Pizza / Code)**

"Hello Server! My _____ is Asha, please send me a study tip!"

Question 3: Multiple Choice - What does React do?

Circle the correct answer below:
What is the main job of **React** in our project?

1. To bake delicious chocolate chip cookies for class
2. To show the text box and button on the screen for the student
3. To turn off the computer when you are done

STUDENT PRACTICE WORKSHEET (CONTINUED)

Question 4: True or False?

Read each sentence below and write **TRUE** or **FALSE** on the line next to it:

a) _____ Flask is used to help run the backend logic.

b) _____ You don't need a browser to see web pages.

c) _____ The "Submit" button helps send your name to get a study tip.

Question 5: Create Your Own Study Tip!

Imagine you are the Flask backend! Write a fun and encouraging study tip for a 4th-grade classmate:

Example: "Hey Alex, take a 5-minute stretch break after reading two pages!"